สรุปสมการ Phase 4: Decision Framework

เอกสารนี้สรุปเฉพาะสมการที่สังเคราะห์ขึ้นใน Phase 4 (ไม่มีอยู่ในตำรามาก่อน) จากผล การวัดจริงใน Phase 2 (Memory Wall) และ Phase 3 (Crossover Point) ของ 3 บอร์ดทดสอบ (Arduino Uno R3, NodeMCU ESP8266, ESP32 DevKit V1)

ที่มา: `report/chapter4_results_th.md` §4.4.2–4.4.3, `results/phase4/decision_criteria.csv`, `analysis/phase4_flowchart.py`

มีสมการทั้งหมด **2 สมการ** ใช้ต่อกันเป็น 2 ขั้นตอนตัดสินใจ

สมการที่ 1 — Memory Wall (n สูงสุดที่บอร์ดรันได้)

```
n_max = floor( log2( B / 8 ) )
```

นิยามตัวแปร

ตัวแปร	ความหมาย
`n_max`	จำนวนคิวบิตสูงสุดที่บอร์ดจำลองได้จริงก่อนหน่วยความจำไม่พอ
`B`	จำนวน byte ของ contiguous free block ที่ใหญ่ที่สุดที่ใช้ได้จริง
`8`	ขนาด `sizeof(Cplx)` (float re + float im = 4+4 byte) ต่อ 1 amplitude

วิธีหา `B`

- **ถ้าวัดได้จริง** (บอร์ด dynamic-allocation ที่มี API เช่น ESP32: `heap_caps_get_largest_free_block`) — ใช้ค่านี้ตรง ๆ
- **ถ้าวัดไม่ได้** (บอร์ด static-allocation ล้วนอย่าง Uno หรือไม่มี API ตรวจสอบ) — ประมาณแบบระมัดระวังด้วย **B ≈ SRAM รวม ÷ 2** (เทียบเท่าลด n ลง 1 คิวบิต จากค่าทฤษฎีดิบ $2^n \times 8 \le \text{SRAM}$)

หลักฐานที่มาของค่าประมาณ "ลด 1 คิวบิต"

บอร์ด	n ทฤษฎี	n วัดจริง	ส่วนต่าง

Uno	8	7	−1
ESP8266	13	12	−1
ESP32	15	13	**−2**

ESP32 ตกมากกว่าเพราะมี WiFi+Bluetooth stack พร้อมกันกินพื้นที่ heap มากกว่า — สรุปเป็นคำเตือนในรายงานว่า **บอร์ดที่มี network/RTOS stack หนักกว่าควรลดค่าประมาณ มากกว่า 1 คิวบิต** ถ้าวัด `B` จริงไม่ได้

สมการที่ 2 — Crossover Point (n ที่ offload เริ่มคุ้มกว่า local)

```
n*(บอร์ดใหม่) ≈ n*(บอร์ดอ้างอิง คลาสเดียวกัน)
              + log2( clock ใหม่ / clock อ้างอิง )
              + ตัวปรับสถาปัตยกรรม
```

นิยามตัวแปร

ตัวแปร	ความหมาย
`n*`	จำนวนคิวบิตที่ offload latency เริ่มต่ำกว่า local execution time
`n*(บอร์ดอ้างอิง)`	ค่า n* ที่วัดจริงของบอร์ดในคลาสเดียวกันที่ใกล้เคียงที่สุด
`log2(clock ใหม่/clock อ้างอิง)`	ผลของความเร็วสัญญาณนาฬิกาต่อความเร็วคำนวณ (ทฤษฎี)
ตัวปรับสถาปัตยกรรม	ส่วนเกินที่ log2(clock ratio) ทำนายไม่ถึง — วัดได้จากข้อมูลจริง

ตัวปรับสถาปัตยกรรม (ค่าที่ใช้ในรายงาน)

เปลี่ยนสถาปัตยกรรมแบบไหน	ตัวปรับ
8-bit → 32-bit (ไม่มี FPU ทั้งคู่)	**+1 คิวบิต**

เพิ่ม FPU ฮาร์ดแวร์ (32-bit เดิม)	**+2 ถึง +3 คิวบิต**

วิธีคำนวณตัวปรับ — เอา Δn* ที่วัดจริง ลบ log2(clock ratio) ทฤษฎีออก ส่วนที่เหลือคือผลจากสถาปัตยกรรมล้วน ๆ:

การเปลี่ยนแปลง	clock ratio	log2(ratio)	Δn* วัดจริง (h_all/bell_ghz/qft)	ส่วนเกิน (สถาปัตยกรรม)
Uno → ESP8266 (8→32-bit, ไม่มี FPU ทั้งคู่)	5×	2.32	+3, +4, +3	**+0.7 ถึง +1.7**
ESP8266 → ESP32 (32-bit ทั้งคู่, +FPU)	3×	1.58	+5, +3*, +5	**+1.4 ถึง +3.4**

* bell_ghz ของ ESP32 ชนกับ Memory Wall (n=13) พอดี ค่าจริงอาจมากกว่านี้ — ตัวเลข +1.4 จึงเป็นขอบล่างที่วัดได้ ไม่ใช่ค่าจริงที่แน่นอน

ยิ่งวงจรมีสัดส่วนคำนวณเลขจุดทศนิยมสูง (เช่น `qft` ที่มีเกท CPhase มาก) ยิ่งได้ประโยชน์ จาก FPU มากกว่าวงจรที่เกทหลักเป็น data-movement (เช่น `bell_ghz` ที่ใช้ CNOT เป็นหลัก)

ข้อจำกัดของทั้ง 2 สมการ (ต้องระบุไว้ทุกครั้งที่อ้างอิง)

1. มาจากการวัดบอร์ดเพียง **3 บอร์ด** (2 คู่เปรียบเทียบ) — ตัวปรับสถาปัตยกรรมเป็นค่า ประมาณหยาบ ไม่ใช่กฎฟิสิกส์
2. Offload latency ที่ใช้คำนวณผูกกับสภาพแวดล้อม PC เครื่องเดียว (macOS, Python 3.11, Qiskit Aer 0.17.2, USB-serial adapter รุ่นที่ใช้ทดลอง)
3. ทดสอบเฉพาะวงจร h_all, bell_ghz, qft — วงจรอื่นที่สัดส่วนเกทต่างออกไปอาจมี ตัวปรับต่างจากนี้
4. Memory Wall formula ที่ใช้ `largest_block` วัดแม่นยำบน ESP32 บอร์ดเดียว (ESP8266 ยังไม่ได้วัด largest_block โดยตรง ใช้ค่าประมาณ −1 คิวบิตแทน)

หมายเหตุความไม่ตรงกันที่ตรวจพบ (ยังไม่แก้ ณ ตอนเขียนเอกสารนี้)

ตัวเลขตัวปรับ FPU ที่แสดงในกล่องข้อความของภาพ `results/phase4/fig_phase4_decision_flowchart.png` (สร้างจาก `analysis/phase4_flowchart.py:79-83`) เขียนว่า **"+3 ถึง +5 คิวบิต"** ซึ่งเป็นเลข Δn^* ดิบที่ยังไม่หัก log2(clock ratio) ออก — ต่างจากตัวปรับที่ถูกต้องตามการคำนวณข้างต้น (+2 ถึง +3 คิวบิต) ถ้านำสูตรในภาพไปใช้ตรง ๆ จะนับผลของ clock ratio ซ้ำสองครั้ง ทำให้ n^* ที่ทำนายสูงเกินจริงประมาณ 1.6 คิวบิต ต้องแก้ตัวเลขในภาพก่อนนำเอกสารนี้ไปใช้อ้างอิงในเล่มจริง